ARCHITECTURE AND FLOWS

PowerGlove Vision Architecture

System boundaries, recognition, tuning, game input, and deployment.

2026 EDITION / IAIN BENNETT / MIT LICENSE

A camera-to-controller system for the **PowerGlove Vision Controller (Arduino UNO Q)** and RetroPie.

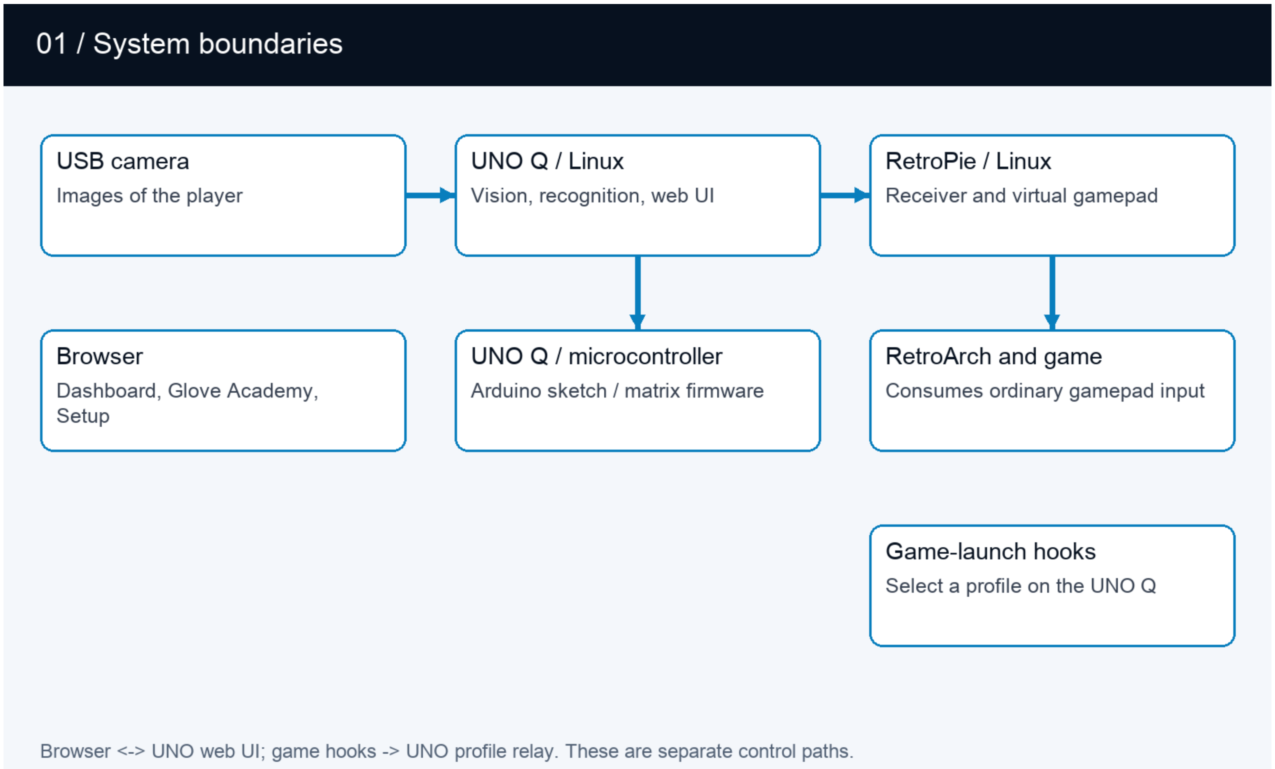
This guide describes the implementation reviewed on September 4, 2026, including three-step tuning, optional personal hand setup, shared gameplay thresholds, the matching Glove Academy/Tune matrix animations, and the verified Arduino sketch build. It is a map of current behaviour, not a proposed redesign or a hardware test report.

Read this first

PowerGlove Vision observes a hand on the PowerGlove Vision Controller, turns its measurements into controller states, and sends those states to a virtual gamepad on RetroPie. The browser configures and explains that process; it is not required in the per-frame gameplay path. The PowerGlove Vision Controller microcontroller drives the status matrix; Linux performs hand tracking and gesture recognition.

There are four independent questions: which game profile is selected, whether the camera is running, whether the player has armed controller delivery, and whether a valid registered-game or manual context currently permits packets. Glove Academy can open the camera while the selected profile is **Gestures off**. Glove Academy and Tune both pause game input. A healthy web page does not by itself establish that the camera, receiver, or game is working.

System boundaries



System boundaries: camera to UNO Linux to RetroPie; separate browser, microcontroller, and game-launch paths

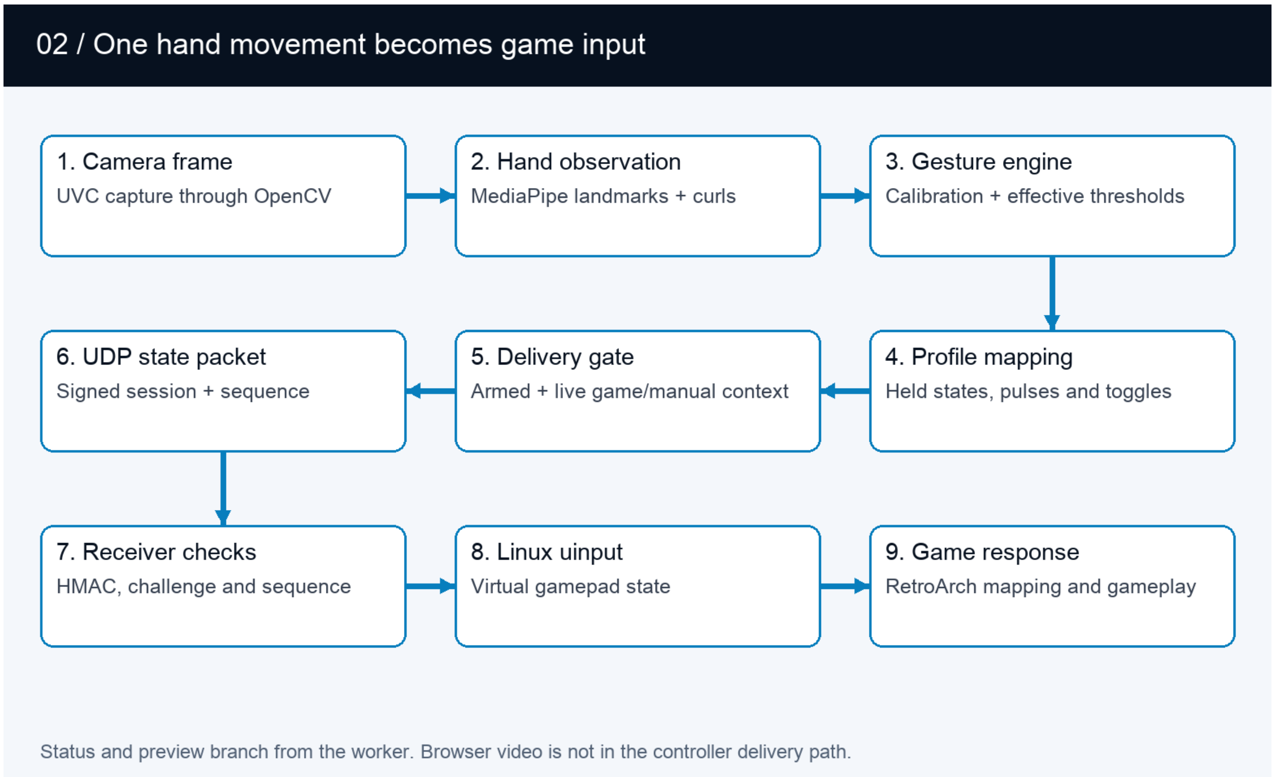
Boundary	Responsibilities	Does not own
Browser	Dashboard, Play, Glove Academy, Tune, Setup, Games, Help; live feedback and user commands	Authoritative per-frame recognition or gamepad output
PowerGlove Vision Controller Linux application	Web server, vision-worker supervision, camera tracking, calibration, thresholds, profile mapping, network sender	RetroArch button consumption
PowerGlove Vision Controller microcontroller	Arduino sketch, Router Bridge commands, LED matrix animations and pairing display	Camera inference or personal thresholds
RetroPie services	Receive controller packets, expose a virtual gamepad, signal game launches, serve paired game-registry edits	Camera processing
RetroArch and game	Consume virtual-gamepad input using emulator and game mappings	Glove Academy/Tune feedback

App Lab starts `python/main.py` in the main application container. This supervisor runs the website and starts an isolated Python 3.12 vision worker with the packaged MediaPipe wheel. It polls worker status, updates the matrix, and retries a worker that stops. The worker's internal HTTP interface is on loopback port 8089; the public website is on 8088, with secure Setup on 8443.

The supervisor passes the private `data/device.json` path to the worker using `--device-config`; the token itself is absent from process arguments. Device configuration mutations are serialized and atomically replace private files. Setup content and browser actions live in `setup_web.py`, separate from HTTP routes. Start/Stop intent has a single pending slot and serialized delivery attempts; the supervisor retries transient failures until the worker handler acknowledges acceptance. A newer explicit request supersedes pending intent. This mechanism does not buffer camera frames or controller state, and acknowledgement does not prove emulator consumption. Receiver socket timeout and last-valid-packet expiry both publish neutral native state and release the virtual gamepad.

Two app-owned support containers provide the profile-control UDP relay and local-hostname resolution. The profile relay publishes port 55356 and forwards packets to the main service without interpreting or authenticating them. The resolver connects application requests to host Avahi through private Unix sockets. These functions are kept separate from camera inference.

Camera-to-controller flow



Nine-stage flow from a camera frame to the game response

1. The camera layer opens a UVC capture source. A dedicated OpenCV capture thread drains it continuously and publishes only the newest frame; older unprocessed frames are superseded rather than queued.
2. MediaPipe identifies the hand landmarks. The tracker produces a `HandObservation`: detection, confidence, timestamp, palm position and scale, wrist roll, and normalized finger curls.
3. The gesture engine compares that observation with the saved neutral calibration and effective thresholds. Directions are relative to the calibrated palm; apparent hand-size change supplies forward/backward movement.
4. Shared activation/release states and held menu poses feed the selected profile's mapping. The result is a `ControllerState`, including buttons, D-pad, axes, finger values, events, sequence, and tracking/calibration metadata.
5. The worker sends the state only if controller delivery is armed, a live registered-game lease or intentional manual Dashboard context exists, and neither practice nor tuning is active.
6. The sender establishes a receiver-issued challenge, then sends bounded HMAC-SHA256 controller datagrams over UDP 55355. Packets contain session and sequence identifiers, never the shared token.

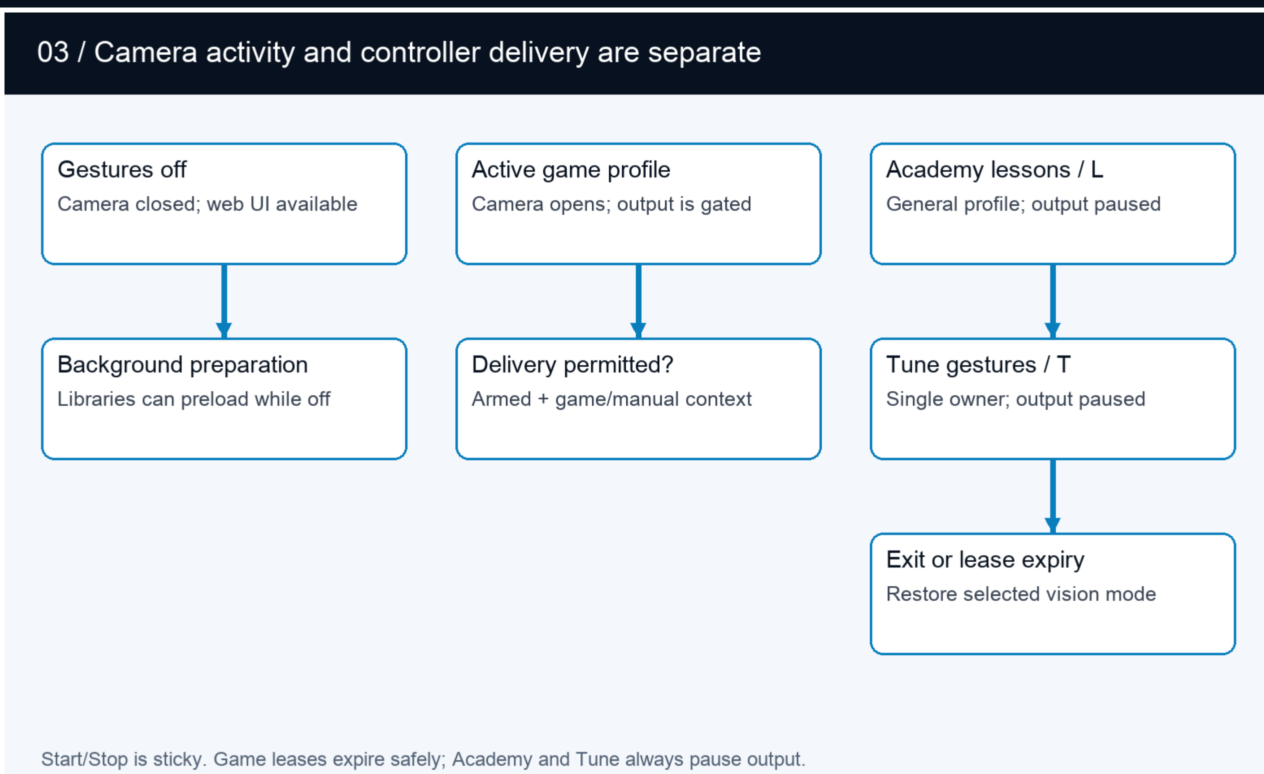
7. The receiver checks the message HMAC, live challenge, peer, and increasing sequence. It creates the real virtual controller when the first accepted packet arrives.
8. Linux `uinput` exposes the virtual gamepad to RetroArch, which applies its configured input mapping before the game consumes it.

The worker also publishes diagnostic state after inference. Browser video is submitted at most five times per second and only while a stream consumer is connected. A separate single-slot worker performs JPEG encoding and discards a superseded preview instead of delaying gameplay. Detailed joint and landmark diagnostics follow that preview cadence; finger geometry itself is calculated once for recognition. Controller sending occurs before optional preview work, so the browser refresh rate is not the controller state update rate. Capture age, inference cadence, skipped frames, preview cost, and send time expose the local stages; none alone is an end-to-end camera-to-game latency measurement.

The read-only `scripts/measure-vision-status.py` collector deduplicates observed inference timestamps and capture sequences. Public status is cached by the supervisor, so even frequent polling observes only a subset of results. The collector separates changing profiles, preview state, and delivery conditions; it does not average overlapping rolling percentiles. Camera exposure, network reception, receiver processing, native core pickup, and physical display delay require separate evidence. See the live baseline procedure.

The current transport is ordinary gamepad emulation. Bad Street Brawler maps Glove Zap to a 180 ms simultaneous Left + Right pulse on each push activation; its FCEUmm game-specific options allow that combination. The receiver already transports both directions. This action does not require native glove packets. Preserved finger and analogue values do not establish native original-Power-Glove support in the emulator. That remains a separate integration concern.

Runtime modes



Runtime modes distinguish camera activity from the controller delivery gate

The worker keeps a lightweight control loop alive while vision is idle. Libraries can preload in the background without opening the camera. Selecting an active profile or opening Glove Academy requests camera/tracker initialization. Slow opens, reads, and cleanup run asynchronously so control requests remain responsive. The website reports startup and recovery rather than treating an empty first frame as completed initialization.

Mode	Vision profile and camera	Controller delivery	Matrix
Gestures off	Camera closed; selected profile off	No gameplay states	Power Glove attract animation
Active profile	Selected game profile; camera requested	Only when armed and a game/manual context is active	Ready/tracking status and profile display
Glove Academy learning	General practice profile; camera requested	Paused	Scanning L
Tune gestures	Practice with selected tuning scope and preview	Paused, including after a game-launch request	Scanning T

Glove Academy preserves the selected game profile while using a mapping-independent practice profile for its sixteen lessons, including **Glove Zap**, **Pull Back**, both wrist rolls, close hand, and menu guard. Progress is saved for each player; completing all sixteen lessons earns the Glove Master award. Browser leases support multiple Glove Academy tabs; the last lease ending restores the selected vision mode. Leases expire after six seconds without refresh. Dashboard also clears abandoned practice sessions. Glove Academy learning restores its prior controller intent; Tune requires an explicit start from Dashboard when finished.

The browser downloads a backup for only the selected player, usually to its Downloads folder. Restore reads that chosen file and updates the selected player after review. The Controller keeps all players in one `data/gesture-tuning.json` store; portable downloads are separate copies and exclude lesson progress. See backup locations.

Tune has a single owning session. Exiting or losing that session discards its recordings and preview, but saved values remain. A game launch can change the selected profile during Tune without allowing game input to escape the delivery gate. Profile/mode transitions release old controls and refresh sender sessions where required so old state does not carry into a new mapping.

The L and T render through the same sketch function: a dim letter, bright scan line, and trailing glow. Eight frames advance every 160 milliseconds. The sketch also handles other status, profile, and pairing indications; matrix activity is feedback about state, not evidence of successful game delivery.

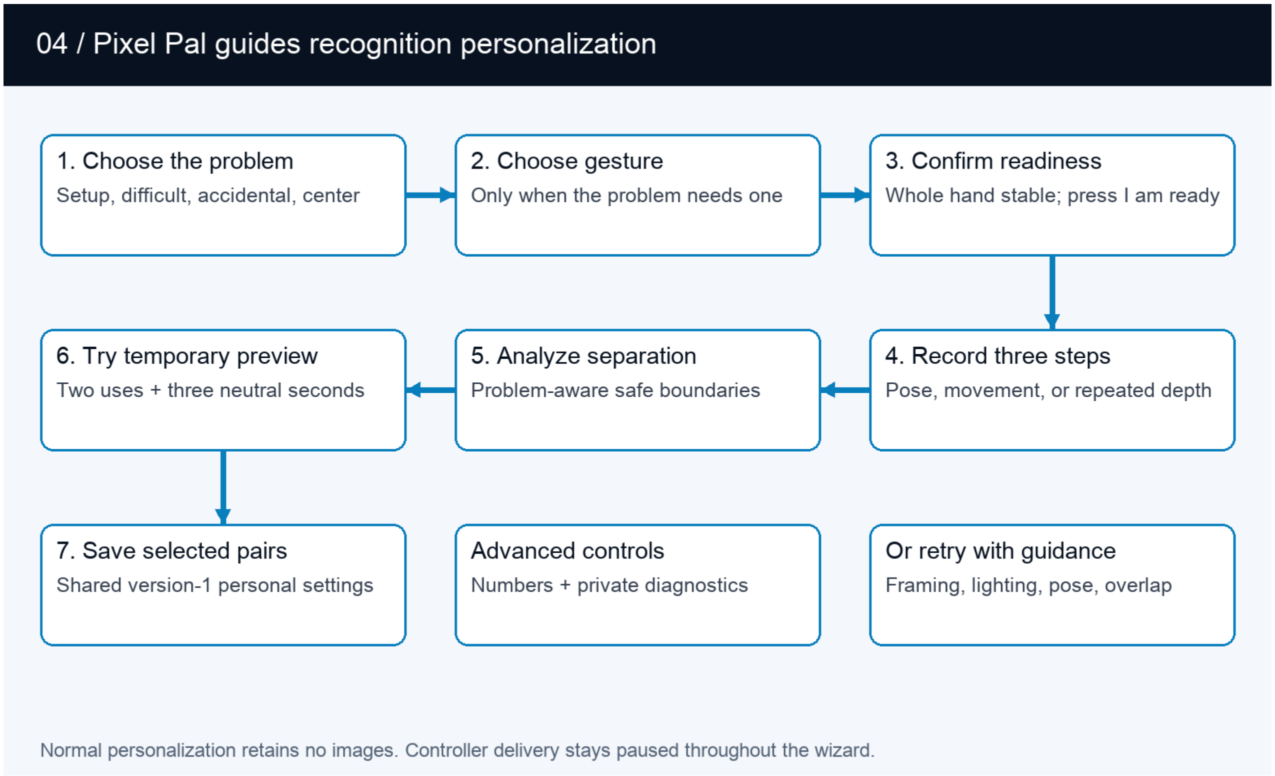
Recognition and tuning

Finger curls use the strongest measured joint bend. The four fingers include the base knuckle; the thumb uses its two outer joints. The tracker prefers MediaPipe world landmarks, with an image-coordinate fallback. Recognition is threshold-based; personal setup does not retrain the MediaPipe model.

A held action has two cutoffs. Crossing **Activation** starts it; returning below the lower **Release** value stops it. This avoids flicker near one cutoff. Finger controls, direction/roll states, Glove Zap, Pull Back, and movement-based mappings use these states. Profile-specific pulses and toggles still determine what the game receives. An indicator remaining active is therefore not a promise of a continuously held game button.

V-sign and thumbs-up also require the correct extended/curled fingers and the deliberate debounce (0.50 seconds for Start and 0.15 seconds for Select), then issue a short menu pulse. Live pose feedback uses the same finger checks. Personal pairs supply the closed-finger activation and extended-finger release boundaries; untouched fingers use the existing menu defaults. A confirmed lesson can remain complete after its brief controller pulse has ended.

Directions, curls, rolls, and Closed Hand are evaluated from every fresh inference result without an additional confirmation timer. Depth actions are motion-confirmed: Glove Zap and Pull Back need two consecutive beyond-threshold observations and at least 0.10 normalized palm-scale movement in the correct direction within 250 ms. Reversal, calibration, profile transition, or tracking loss discards an unfinished candidate. Confirmed actions retain their existing hysteresis and profile-specific output semantics.



Personalization flow: choose a problem, record guided phases, pass a preview test, and save

Tuning scope	First recording	Middle recording	Final recording
Set up a new hand	Comfortable open hand	Gentle fist, thumb curled outside fingers	Comfortable open hand
Finger/menu pose	Comfortable open hand	Selected gesture held steadily	Comfortable open hand
Glove Zap	Open hand at starting distance	Three pushes and returns	Return to starting distance
Pull Back	Open hand at starting distance	Three pull-backs and returns	Return to starting distance
Direction or wrist roll	Starting position and wrist angle	Selected movement held steadily	Return to starting position and angle

Pose, direction, and roll recordings last two seconds; the repeated depth-motion step lasts six. Recording is enabled after a calibrated hand at 70% confidence has remained completely inside the image for one second. A user-controlled two-second countdown precedes every sample.

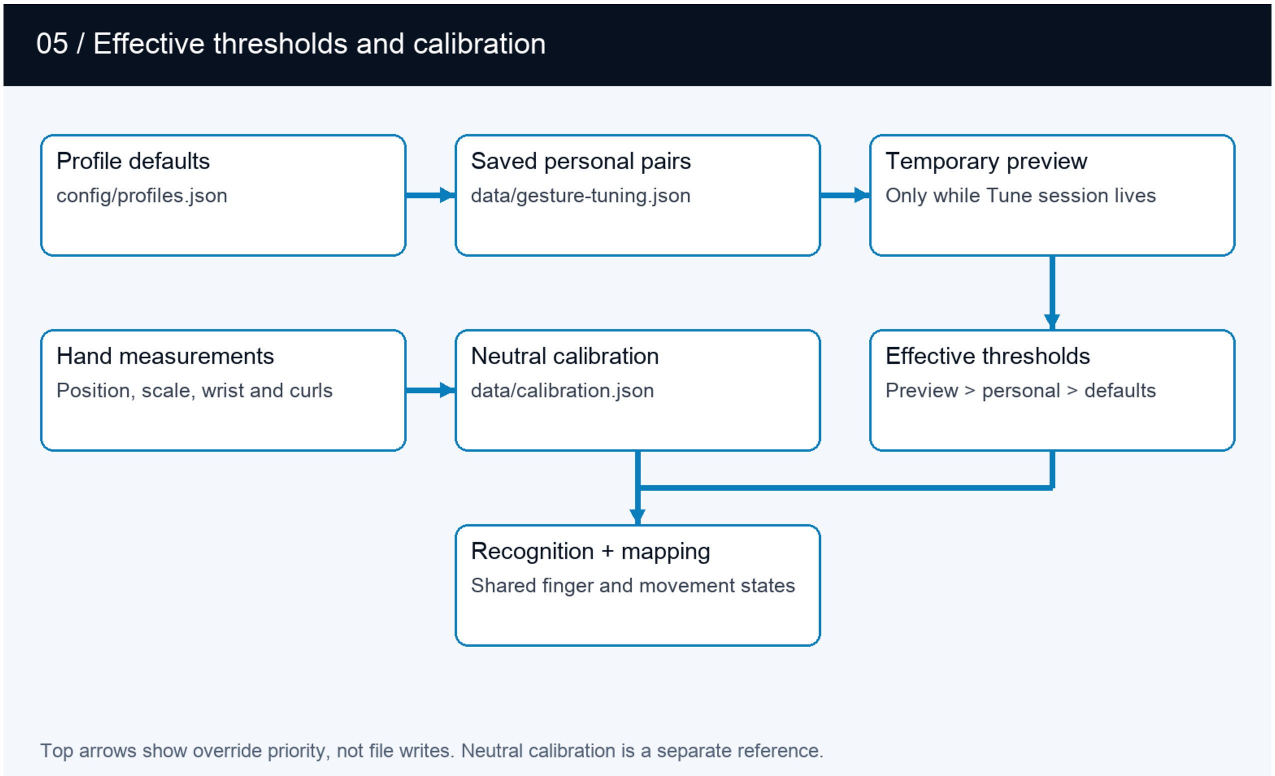
Each step needs at least twelve accepted samples. The manager accepts calibrated, detected hands with confidence at least 0.7, rejects repeated frames and non-finite measurements, and caps samples per recording. Tracking gaps contribute no samples; too few samples require a retry. Neutral calibration changes invalidate recordings and previews.

For each adjusted component, analysis compares the 95th percentile of both open/rest phases with the performed phase. It requires a gap of at least 0.08. Standard setup places activation/release at 65%/30% of the gap; difficult and accidental paths use 55%/30% and 75%/40%. Repeated depth motion uses its upper quartile so returns to neutral are not misread as failed movement.

Hand setup observes both states for all five fingers. Individual gesture tuning can run without it. Fingers extended throughout retain their existing settings; extended-only samples cannot establish a curled boundary. Automatic suggestions now check all required fingers against the candidate configuration using the same pose checks as recognition. At least 90% of accepted samples must match the complete pose simultaneously. The opening and release phases must also show all selected fingers extended in at least 90% of samples. A failure names the finger and phase; no suggestion is retained. Strong curls cannot compensate for fingers that should be extended. Thumbs-up checks a straight thumb and four curled fingers; it does not impose an upward screen direction. Manual threshold edits validate range and scope, not recorded pose quality. Live testing is still needed.

The candidate is temporary until the same recognition path observes two complete activation/release cycles and three neutral seconds. Only then can the wizard atomically merge selected pairs into the active player's version-4 record. Raw controls remain inside Advanced. Normal personalization retains no camera recording. The separate diagnostic path deletes its temporary AVI after producing an aggregate-only report.

Profile and configuration flows



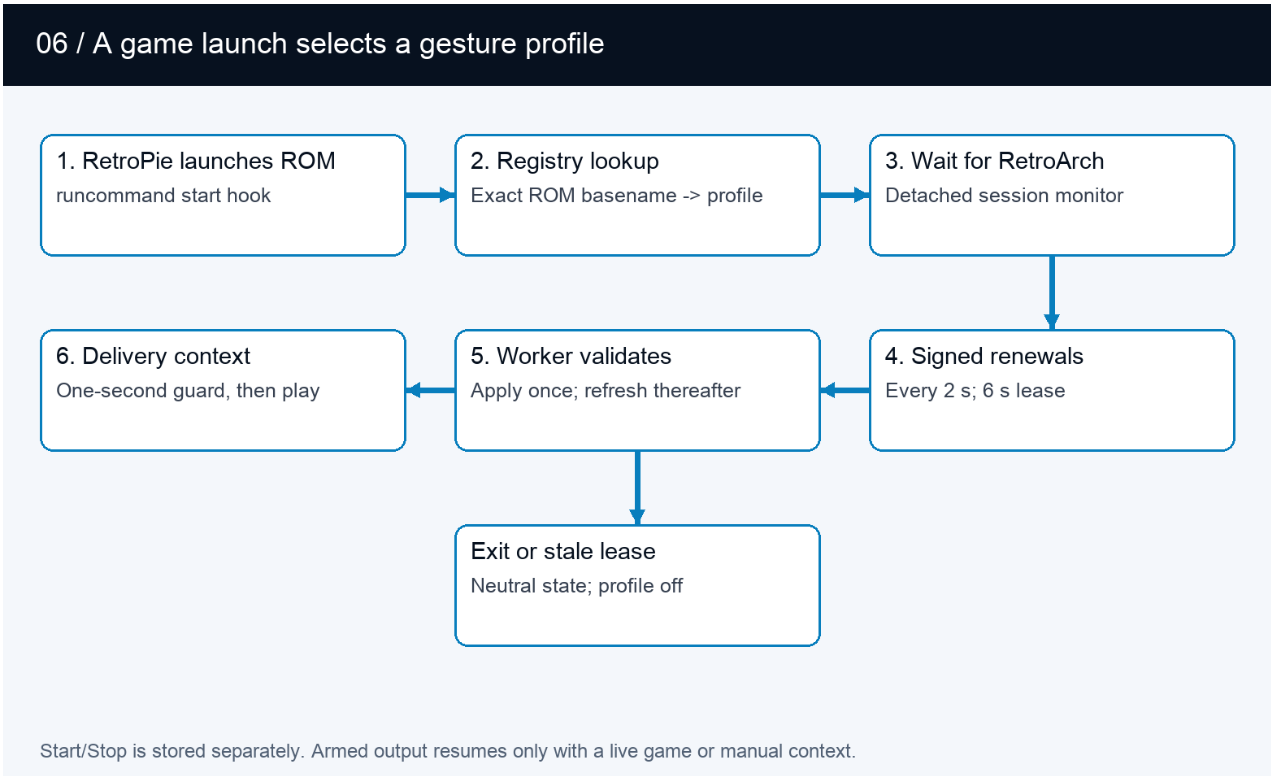
Threshold precedence and the separate neutral-calibration reference

Effective settings are resolved component by component: shipped shared recognition defaults, then the active player's saved overrides, then temporary Tune preview. The gesture engine receives the resulting configuration during frame processing, so saved values also apply when controlling a game. Adjusting a finger changes other gestures that use that finger; it does not change the button assignments in a game profile.

Data	Owner and lifetime	Purpose
`config/profiles.json`	Shipped project source	One shared set of recognition parameters; profiles remain output mappings

Data	Owner and lifetime	Purpose
`data/gesture-tuning.json`	PowerGlove Vision Controller, persistent	Version-4 player presets, per-player calibration, sensitivity, Academy progress, and required-center flag; versions 1-3 migrate with a backup
`data/calibration.json`	PowerGlove Vision Controller, private persistent	Neutral palm position, apparent scale, wrist angle, and positional jitter for the installed camera and player
`data/device.json`	PowerGlove Vision Controller, private persistent settings	Destination, selected settings, pairing-related configuration
Tuning samples, preview, leases	Worker memory only	Temporary measurement and ownership state
`config/games.json`	Shipped default registry	Exact ROM-name mappings copied to the RetroPie installation
RetroPie registry and launcher settings	RetroPie, persistent	Active game-to-profile mappings and UNO destination
`data/models/hand_landmarker.task`	PowerGlove Vision Controller, verified cache	Reusable pretrained hand-landmark model

Neutral calibration is distinct from hand setup. It accepts 24 detected hand observations at 70% confidence or better, centers position, depth, and roll, records 95th-percentile X/Y jitter, and lets movement thresholds rise only when needed to remain safely above that noise; hand setup establishes finger thresholds. The app reuses valid neutral calibration across Glove Academy, profile changes, camera reconnects, and worker restarts. Recalibrate after moving the camera or changing playing position. Ordinary updates preserve `data/` rather than replacing it with example configuration. The portable release baseline is `config/profiles.json`; raw neutral coordinates are deliberately machine- and player-local.



Profile-selection flow from RetroPie launch hook through the UNO relay and worker

At game launch, the RetroPie hook looks up the exact ROM basename. For a registered game it records a user-owned session marker, starts a detached monitor, and waits for RetroArch to exist before sending input context. The monitor sends a signed profile renewal every two seconds to UNO UDP 55356 while both RetroArch and the marker remain active. Each renewal carries a bounded six-second lease. The app-owned relay forwards the bytes to the worker; the worker authenticates them and treats repeated renewals as lease refreshes rather than profile transitions. The acknowledgement travels back through the relay. The relay has no shared token and cannot declare a profile applied.

The first live renewal changes profile once and starts a one-second initialization guard. A PowerGlove Vision Controller application restart can therefore rediscover an already-running registered game from the next renewal without exposing the runcommand menu to hand input. Game-end hooks, RetroArch termination, marker replacement, unknown games, and lease expiry request or produce neutral/off state. The player's armed/stopped choice is stored separately: Stop remains sticky, while armed alone never authorizes output. Manual Dashboard profile selection provides an explicit testing context without pretending that a registered game is running. Unsupported or unregistered games do not gain a mapping merely because their filenames resemble a registered title.

Setup's Games editor uses a separate path: browser to UNO web API, then the paired UNO proxy to the RetroPie Games service on TCP 55358. Challenge/HMAC exchanges protect registry operations; revision checks prevent stale edits and atomic replacement preserves a previous valid copy. Saving a registry mapping affects the next launch; it does not rewrite the running game's mapping immediately.

Optional native Super Glove Ball path

The supported FCEUmm path consumes the same virtual gamepad as every other game. For native research, the authenticated RetroPie receiver also publishes a versioned, fixed-size latest-sample record in `/run/powerglove/native-state`. The separately built `lr-nestopia-powerglove` core maps that file read-only, copies at most one coherent current sample per emulated frame, and adds no queue or smoothing. Invalid, stale, uncalibrated, lost, or wrong-profile samples leave the emulated glove neutral.

Exact-ROM traces now confirm the ten-byte packet boundary, MSB-first reads, native Start, continuous X/Y, signed Z, and open/fist/index packet response. Live full-game play confirms the resulting grab/throw, index-fire, and fist-plus-forward Power Punch actions. A matched same-ROM test confirms that FCEUmm requests only ordinary joypad input while both cores visibly respond to all four directions by frame 3. Stale, uncalibrated, lost, and wrong-profile samples produce a neutral packet. The shared layer publishes its five-finger closed-hand and index-point decisions explicitly so the core does not reconstruct compound poses from partial finger data. Native wrist rotation and remaining unused packet fields stay evidence-gated. They are outside the Super Glove Ball actions confirmed in live play. Stock Nestopia remains untouched; the custom core is enabled only through a Super Glove Ball per-ROM emulator choice after it is built locally from pinned GPLv2 source and verified on the cabinet. The ordinary release carries the patch and build recipe, not a compiled core. See the native compatibility record.

Interfaces and recovery

Setup's four status markers share the matrix's cached app, console-service, authenticated-response, and independent Networking checks. The read-only `/api/connection-status` endpoint requests bounded background refreshes; no network probe runs on the capture or controller-send path. Unknown or expired results are shown in grey. Reachability and authentication do not establish emulator consumption. See Setup status.

Player operations pass through the bounded same-origin `/api/players` endpoint into the worker. Its tuning lock owns one atomic player/settings/progress file. Generations reject stale writes. Each player retains a saved calibration; switching requires fresh centering or explicit same-position reuse. Version-2 portable backups include personal and effective sensitivity, source software identity, name, and the player's neutral reference. They exclude credentials and Academy progress. Version-1 portable backups are rejected; earlier version-2 files remain supported. A version-4 player store journals confirmed calibration reuse; the worker writes `calibration.json` before clearing the pending reference and centering gate. Output remains paused until Start controller. The journal resumes after crashes; internal version-1/2/3 stores migrate with recovery backups and unchanged progress. Progress writes occur on lesson transitions, not frames.

Hostname resolution for controller sends runs in one background thread with a single cached address. No controller states are retained by that thread. Missing or expired addresses cause the current send to be skipped; later calls use their own newest state. Host physical Wi-Fi/Ethernet link health is sampled independently by an unprivileged systemd timer, which publishes a small expiring JSON record for the supervisor and fourth Off-mode matrix pixel. Console reachability remains a separate probe.

Build metadata records the source commit and candidate. A generated sketch fingerprint is compiled into firmware and read through Router Bridge in the supervisor, independently of the expected packaged value. Missing readback stays unavailable; this introduces no firmware RPC in the vision worker's frame path.

Interface	Direction	Contract
HTTP 8088	Browser to PowerGlove Vision Controller	Pages, live status/video, ordinary settings and commands
HTTPS 8443	Browser to PowerGlove Vision Controller	Secure Setup and pairing workflow
HTTP 8089, loopback	Supervisor/web proxy to worker	Internal status, frame and control requests
UDP 55355	PowerGlove Vision Controller to RetroPie	Signed controller states, session, challenge, and sequence; handshake replies return to the sender socket
UDP 55356	RetroPie to UNO relay to worker	Signed profile requests and acknowledgements
`/run/powerglove/native-state`	Authenticated RetroPie receiver to custom core	Read-only, guarded latest sample for experimental native input
TCP 55357	Pairing participants	Temporary one-time-code pairing service
TCP 55358	PowerGlove Vision Controller to RetroPie	Paired game-registry service
Private Unix sockets	App resolver to host Avahi	Local hostname resolution
Router Bridge RPC	Linux supervisor to microcontroller	Matrix status/profile/pairing commands

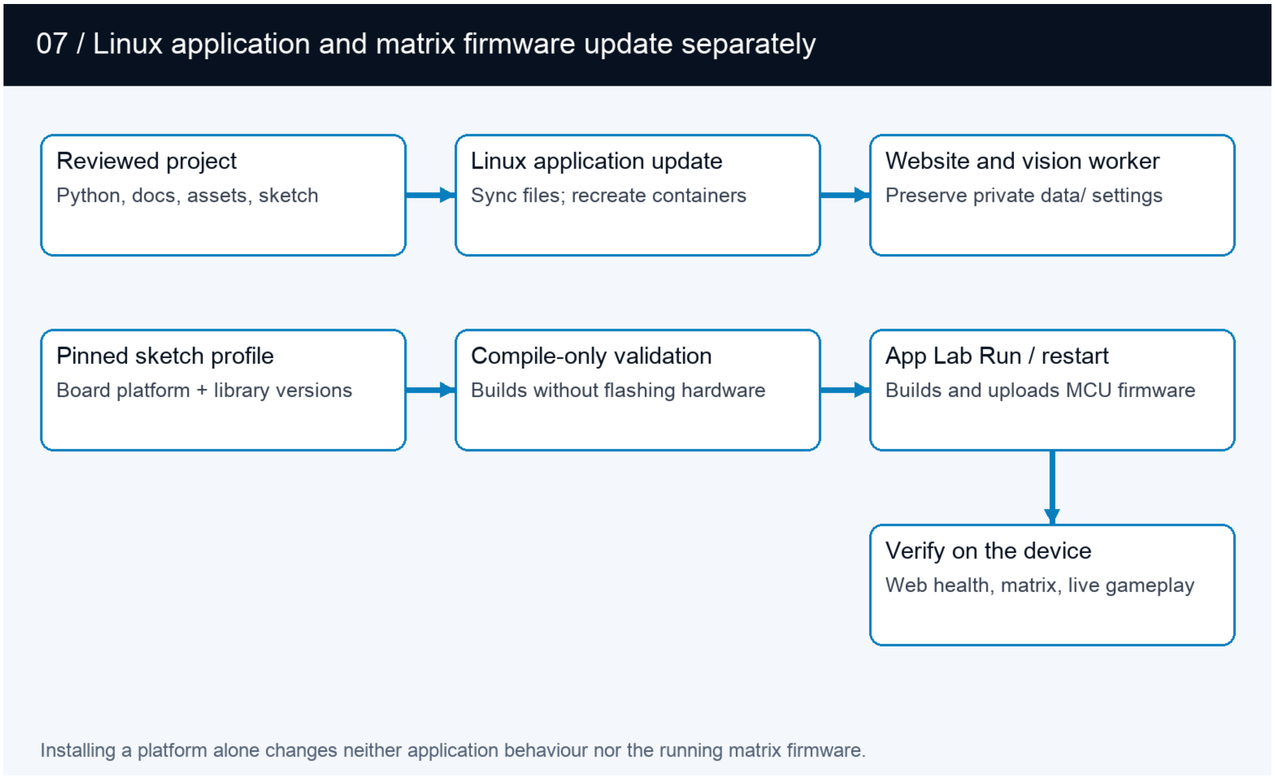
The LAN remains a trust boundary. Controller version 2 uses its own domain-separated HMAC-SHA256 and receiver-issued challenges. It does not encrypt input. Legacy version-1 input is disabled by default and never emitted by the new sender. Do not describe all links as equivalent secure channels. Pairing and registry exchange have their own protections; browser mutations use the existing request-header and Origin checks. See the Security policy for the full trust model.

Failure or transition	Implemented response	Interpretation
Hand tracking lost	Engine clears held states after its loss delay	Stops stale recognized actions; camera recovery is separate
Controller packets stop	Receiver releases controls on socket timeout, default 250 ms	A receive timeout, not a measured end-to-end acknowledgement
Hostname or UDP send failure	Sender reports error and throttles retries	Vision and local practice can continue
Camera open/read failure	Worker reports starting/error and retries asynchronously	Healthy website can coexist with unavailable vision

Failure or transition	Implemented response	Interpretation
Worker exits	Supervisor reports failure and retries	Temporary in-memory Tune state is lost
Tune browser disappears	Six-second lease expires	Preview and recordings discarded; saved pairs retained
Calibration changes	Current Tune recordings/preview invalidated	Record new measurements against the new reference
Shutdown requested	Web action writes fixed request; host systemd helper requests halt	The tested PowerGlove Vision Controller can restart; not proof that power is safe to remove

A successful UDP send means the local networking call succeeded. It does not prove the receiver applied a state or the game accepted it. Diagnose in stages: hand detected, measured values, recognized/held action, delivery gate, sender error, receiver/gamepad state, then emulator/game mapping.

Build and deployment



Separate Linux application and microcontroller firmware deployment paths

The versioned `install-uno-q.sh` and `install-retropie.sh` entry points download matching packages and call the shared host installer. The UNO route uses App Lab CLI to build/upload the sketch and start the app; it installs both startup and fixed-purpose shutdown/camera-recovery helpers. The RetroPie route installs the receiver and launch integration, then checks emulator and registered-game configuration.

There are two deployable parts. Python, website, documentation, assets, and service support run on Linux. The **Arduino sketch** is the microcontroller source code; its compiled and installed version is the **matrix firmware**. That firmware drives the LED matrix and handles Router Bridge commands. The Wi-Fi deployment script synchronizes Linux application files and recreates containers; it does not upload matrix firmware. A documentation-only sync can serve new Markdown and

PDFs without restarting the application, provided Python route registration has not changed.

The Arduino sketch currently depends on the Arduino Zephyr platform **1.0.0** for `arduino:zephyr:unoq`. Zephyr is the current platform dependency, rather than the name of the PowerGlove component. The build configuration also pins:

- Arduino_RouterBridge **0.4.3**
- Arduino_RPClite **0.3.0**
- ArxContainer **0.7.0**
- ArxTypeTraits **0.3.2**
- DebugLog **0.8.4**
- MsgPack **0.4.2**

The verified platform supplies Arduino_LED_Matrix **0.1.3**. Retain the complete project `sketch/sketch.yaml` when synchronizing with App Lab.

Installing that platform makes build tools available. Compile-only validation builds against it but does not flash hardware. App Lab **Run**, or its supported app-restart command, compiles the Arduino sketch and uploads the matrix firmware. Back up the installed source and firmware cache, verify compilation, upload, then check application health, bridge response, physical matrix appearance, and actual controls. Keep private settings intact. Detailed commands are in the Installation Guide.

Documentation has an editable Markdown source, generated diagrams, built-in Help rendering, and a PDF edition. `scripts/build-architecture-diagrams.py` regenerates these seven figures. `scripts/build-docs-pdf.py` generates the PDF set. The Help and package allowlists explicitly include this architecture guide. The local quick reference remains excluded from public deployment.

Implementation map

Paths below are relative to the project root. This map identifies responsibility; it does not claim every path has been independently security-audited.

Responsibility	Start reading here
Supervisor, worker launch, matrix ownership	`python/main.py`
Camera lifecycle and frame-to-send loop	`src/powerglove_vision/vision_app.py`
Capture selection and landmark measurements	`src/powerglove_vision/camera.py`, `tracker.py`
Observation/state data objects	`src/powerglove_vision/model.py`
Calibration, thresholds, held gestures, mappings	`src/powerglove_vision/gesture.py`
Recording, suggestions, previews, persistence	`src/powerglove_vision/tuning.py`
Public HTTP routing and worker proxy	`src/powerglove_vision/control_server.py`
Shared page shell and maintained browser modules	`web_common.py`, `dashboard_web.py`, `academy_web.py`, `games_web.py`, `tuning_web.py`, `setup_web.py`, `player_web.py`

Responsibility	Start reading here
Worker requests, status, practice leases	`src/powerglove_vision/debug_server.py`
Controller packets and virtual gamepad	`src/powerglove_vision/transport.py`, `controller_protocol.py`, `receiver.py`
Profile requests, launch hooks, UDP relay	`src/powerglove_vision/profile_control.py`, `retropie_hook.py`, `scripts/profile-relay.py`
Paired Games editing	`src/powerglove_vision/game_registry.py`
Pairing and hostname resolution	`src/powerglove_vision/pairing.py`, `python/ssh_pair.py`, `src/powerglove_vision/resolver.py`
Matrix translation and firmware	`src/powerglove_vision/matrix.py`, `sketch/sketch.ino`
App services and installation	`app.yaml`, `bricks/local/`, `scripts/setup-machine.py`
Help and printable guides	`src/powerglove_vision/help_content.py`, `scripts/build-docs-pdf.py`

Validation boundaries

Code inspection establishes the flows described here. The prior work also compiled the Arduino sketch and deployed the matrix firmware, checked application health and bridge responses, and verified saved configuration preservation. Those checks are different from visually observing the physical matrix or validating real hands in a running game.

Before releasing recognition changes, exercise optional hand setup and individual tuning without setup; V-sign and thumbs-up with different curl ranges; incorrect extended fingers; incomplete releases; tracking loss; insufficient/overlapping samples; calibration changes; preview expiry; persistence; and reset. Confirm that feedback agrees with recognition and output remains paused throughout Glove Academy/Tune. Complete live camera and gameplay tests before describing a reduced recording count as validated for other users.

Matrix during startup

The Arduino sketch shows an hourglass before its blocking Router Bridge setup. A dedicated display task owns subsequent framebuffer writes and keeps startup feedback moving independently of Linux and Python initialization. The main sketch task registers the bridge endpoints; those endpoints update requested status/profile values, and the display task renders them. If the display-task stack allocation fails, the first hourglass stays visible during setup and the normal sketch loop takes over rendering afterward. This task currently uses the Zephyr API supplied by the Arduino sketch platform.

Python requests loading before importing the web controls, then forwards normal worker status. The hourglass indicates activity, not measured completion. It does not replace the protected system boot display. The optional host user service `powerglove-early-start.service` releases the installed sketch earlier using the loader release flag, after checking the selected app and sketch samples. It never resets, halts, or flashes the sketch. This brings the existing hourglass forward while App Lab continues starting. Failure falls back to normal App Lab startup; the cold-boot trial was confirmed on the physical board.

Idle display preferences

The supervisor passes the persisted `matrix_attract` setting to the sketch through `set_powerglove_attract(mode, connections)`. Only `PG_GESTURES_IDLE` consumes it; there is no global brightness change. In Off mode a bounded background probe checks TCP reachability and authenticates the existing RetroPie Games service. The supervisor publishes cached indicator bits; capture, recognition, transport, T/L displays, and game-state paths are unchanged.

Signed controller session lifecycle

The nonblocking sender emits a signed hello with random session and request identifiers. RetroPie replies with a fresh random 128-bit challenge; only a signed reply matching the sender's current request, session, and configured receiver port is accepted. On Linux, receiver replies preserve the destination address and receiving interface using IP_PKTINFO, so Ethernet/Wi-Fi multihoming works through container NAT. The sender also permits a different source address when HMAC, request, session, and port match. A valid state activates that challenge. Activation invalidates every older active and pending challenge; subsequent states require increasing sequence numbers. A replayed hello can obtain a new challenge but cannot supply an authenticated state for it. Receiver restarts discard all challenges, so recorded traffic from a previous process cannot activate input.

At most eight pending handshakes are retained, for three seconds each. No input state is retained while negotiating. Hellos repeat every 250 milliseconds before the first challenge, then once per second to recover a receiver restart. The sender reads at most eight replies per update without blocking and sends only that update's state. Periodic handshake traffic does not reset the receiver's input-release deadline. Both native-state publication and uinput remain behind the same accepted-state check; the core and recognition paths are unchanged.

Dashboard and Academy now import their maintained pages from separate modules. Games and personalization have their own modules, and `web_features.py` preserves the existing import surface without obsolete UI definitions. The extracted Dashboard, Academy, Play, and Setup pages are byte-for-byte identical to the previous output.

For a guided symptom check, see [Troubleshooting by symptom].